TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Môn học: **Lập trình Đa nền tảng**

«Chủ đề 01: OOP nâng cao & Xử lý ngoại lệ»

Module quản lý dữ liệu an toàn, loại bỏ crash và cách ly lỗi trong Dart

Sinh viên thực hiện:
1. Lê Anh Khoa
2. Ngô Quang Thế

Đà Nẵng, 2026

Mục lục Nội dung Báo cáo

PHẦN I: NỀN TẢNG OOP

Thuyết trình: Lê Anh Khoa

- **Đặt vấn đề & Phân công công việc (WBS)**
- **4 Trụ cột OOP trong Dart** & Lớp bất biến (*Immutable Object*)
- **Constructor:** Named & Redirecting Factory
- **Factory Constructor** & Bảng đối sánh
- **Mixin:** Tái sử dụng tính năng, cơ chế on và lý do dùng `with`
- **Generics:** Reified Generics & Bounded Type (`<T extends X>`)
- **Sealed Class & Pattern Matching (Dart 3)**

PHẦN II: NGOẠI LỆ & KIẾN TRÚC

Thuyết trình: Ngô Quang Thế

- **Exception vs Error:** `try-catch`, `rethrow`, `StackTrace`
- **Cây phân cấp Custom Exception** đa tầng
- **Hàm bao bọc (errorWrapper)** & Result Pattern
- **Lưới an toàn toàn cục** (Global Safety Net)
- **Kiến trúc Phân tầng, State Management & Chống Race Condition**
- **Sơ đồ Luồng dữ liệu** (Data Flow Swimlanes)
- **Kịch bản Thực nghiệm, Live Demo & Tổng kết**

Đặt vấn đề, Mục tiêu & Bảng phân công công việc (WBS)

1. Đặt vấn đề & Mục tiêu đề tài:

- **Thực trạng:** Ứng dụng di động dễ gặp sự cố crash do ngoại lệ bất đồng bộ, JSON sai định dạng, hoặc Race Condition khi người dùng thao tác dồn dập.
- **Mục tiêu:** Áp dụng OOP nâng cao và kỹ thuật xử lý lỗi tập trung trong Dart để xây dựng module đặt vé tự phục hồi, cách ly lỗi hoàn toàn.

2. Bảng phân công công việc chi tiết (Work Breakdown Structure):

TT	HẠNG MỤC PHỤ TRÁCH	FILE MÃ NGUỒN CHÍNH	NGƯỜI THỰC HIỆN
1	**UI Design & Safety Net**	`demo_page.dart`, `main.dart`	Ngô Quang Thế
2	**Business Logic**	`booking_service.dart`, `lab_location.dart`	Lê Anh Khoa
3	**State Management & Utils**	`booking_controller.dart`, `result.dart`	Ngô Quang Thế
4	**Database / Mock API**	`booking_model.dart`, `app_exception.dart`	Lê Anh Khoa
5	**Reusable Logic (Mixin)**	`validation_mixin.dart`	Lê Anh Khoa
6	**Báo cáo & Slide Beamer**	Slide LaTeX, Kịch bản kiểm thử Live Demo	Cả hai thành viên

Lập trình Hướng đối tượng (OOP) nền tảng trong Dart

1. **Bốn trụ cột OOP trong Dart:**
   - **Đóng gói (Encapsulation):** Ẩn giấu dữ liệu bằng tiền tố _ (phạm vi thư viện *library-level*, không phải cấp class như Java/C++).
   - **Kế thừa (Inheritance):** Đơn kế thừa qua `extends`, mở rộng khả năng qua `implements` (mọi class đều ngầm định là một interface).
   - **Đa hình (Polymorphism):** Ghi đè phương thức (`@override`) và định kiểu linh hoạt qua lớp cơ sở.
   - **Trừu tượng (Abstraction):** Khai báo qua `abstract class`, định nghĩa khung hợp đồng nghiệp vụ.

2. **Thiết kế Lớp bất biến (Immutable Object):**
   - **Khái niệm:** Đối tượng không thể bị thay đổi trạng thái sau khi đã được khởi tạo thành công.
   - **Thực thi trong Dart:** Khai báo tất cả thuộc tính là `final` và sử dụng `const constructor`.
   - **Lợi ích:** An toàn khi truyền giữa Service và Controller, dễ suy luận trong luồng bất đồng bộ (*async/await*), tối ưu RAM qua chia sẻ thể hiện (*Canonicalization*).

Ứng dụng trong dự án

`BookingModel` được thiết kế bất biến hoàn toàn, giúp luồng dữ liệu giữa Service và Controller luôn toàn vẹn, tránh biến đổi ngoài ý muốn.

Constructor: Named & Redirecting Factory Constructor

1. **Named Constructor (Khởi tạo định danh):**
   - Dart **không hỗ trợ Overloading constructor** cùng tên như Java/C++.
   - Giải pháp: Đặt tên rõ nghĩa theo cú pháp `ClassName.tênRiêng()`.
   - **Initializer List** (`: field = val`): Một trong các cách gán giá trị cho trường `final` trước khi thân hàm constructor thực thi.

```
// Named constructor: initialize empty default
     instance
const BookingModel.empty()
    : id = 'N/A',
      amount = 0,
      type = TicketType.standard,
      labEvent = 'Chưa có thông tin';
```

2. **Redirecting Factory Constructor:**
   - Dùng từ khóa `factory` kết hợp = trỏ trực tiếp đến constructor của lớp con tương ứng.
   - Che giấu chi tiết cài đặt của lớp con phía sau giao diện lớp cơ sở.

```
// Redirecting factory constructor: forward to
     subclass
sealed class Result<T> {
  const Result();
  factory Result.success(T data)
     = Success<T>;
  factory Result.failure(AppException error)
     = Failure<T>;
}
```

Factory Constructor & Bảng so sánh các Constructor

1. Bản chất của Factory Constructor:

- **Không tự động tạo instance:** Bắt buộc phải có lệnh `return` trả về một đối tượng.
- Có thể trả về đối tượng tái sử dụng (Cache / Singleton) hoặc trả về thể hiện của một lớp con.
- Chứa logic nghiệp vụ, thẩm định dữ liệu và chủ động ném ngoại lệ (`throw`).

```
// BookingModel.fromJson (phiên bản minh họa rút gọn)
factory BookingModel.fromJson(Map<String, dynamic> json) {
  if (json case {'id': final id, 'amount': int amount}) {
    return BookingModel(id: '$id', amount: amount, ...);
  }
  throw const DataFormatException('Dữ liệu máy chủ bị lỗi');
}
```

2. Bảng so sánh 3 loại Constructor:

TIÊU CHÍ	NAMED	FACTORY	REDIRECT
Tự tạo instance	Có	Không tự động (cần `return`)	Không
Dùng `this` / Init list	Có	Không	Không
Trả về lớp con	Không	Có	Có
Chứa logic	Có (hạn chế)	Có	Không

Mixin: Tái sử dụng tính năng không qua kế thừa

1. Khái niệm, Linearization & Đơn kế thừa:

- **Bản chất:** Tái sử dụng mã nguồn giữa các class không cùng cây phả hệ.
- **Cơ chế Linearization:** Dart giải quyết xung đột phương thức bằng **Linearization** (thứ tự khai báo từ trái sang phải: mixin sau ghi đè mixin trước), giải quyết xung đột hình thoi (*Diamond Problem*).
- **Từ khóa on:** Giới hạn kiểu được phép dùng mixin (`mixin M on SuperClass`), cho phép gọi `super.method()`.
- **Lý do dùng `with`:** Dart là ngôn ngữ **đơn kế thừa**. `BookingController` đã `extends ChangeNotifier` nên **bắt buộc dùng `with`** để nhúng `ValidationAndLogMixin`.

2. Bảng đối sánh & Code minh họa trong dự án:

ĐẶC TRƯNG	EXTENDS	IMPL	WITH
Bản chất	"Là một"	"Hợp đồng"	"Thêm hành vi"
Số lượng	Duy nhất 1	Nhiều	Nhiều
Thừa hưởng	Có	Không	Có
Có constructor riêng	Có	Có	Không

```
mixin ValidationAndLogMixin {
  void validateTicketCount(int count) {
    if (count <= 0) {
      throw const BusinessLogicException(
          'Số lượng vé phải lớn hơn 0');
    }
  }
  void logInfo(String tag, String msg) =>
      debugPrint('[INFO] [$tag]: $msg');
}
```

Generics: Đảm bảo an toàn kiểu & Reified Generics

1. Mục đích và Ý nghĩa:

- Cho phép viết code tổng quát hoạt động trên nhiều kiểu dữ liệu mà vẫn đảm bảo **an toàn kiểu lúc biên dịch** (*Compile-time Type Safety*).
- Loại bỏ thao tác ép kiểu thủ công không an toàn (`as Type`), phòng ngừa phát sinh lỗi `TypeError` khi chạy.

2. Cơ chế Reified Generics trong Dart:

- Khác Java (*Type Erasure*), Dart **lưu thông tin kiểu generic cả trong lúc chạy** (*Runtime*).
- Cho phép kiểm tra kiểu chính xác lúc thực thi: `result is Result<BookingModel>`.

3. Ràng buộc Generics (`<T extends X>`):

- **Bounded Type:** Giới hạn kiểu tham số nhận lớp con của kiểu định sẵn.
- **Non-nullable Bound:** Với Null Safety, `<T>` mặc định là `Object?`. Viết `<T extends Object>` nhằm **loại trừ triệt để kiểu null**.

```dart
// 1. Non-nullable Bound: loại trừ kiểu null (T extends Object)
T requireValue<T extends Object>(T? v) =>
    v ?? (throw const UnknownException('Giá trị null'));

// 2. Bounded Type: chỉ nhận lớp con của AppException
class ErrorHandler<E extends AppException> {
  final E error;
  const ErrorHandler(this.error);
  void log() => print('[${error.code}] ${error.message}');
}
```

→ *Ứng dụng trong dự án: `Result<T>`, `errorWrapper<T>`*

Sealed Class & Pattern Matching (Dart 3)

1. Tính năng Sealed Class trong Dart 3:

- `sealed class Result<T>`: Khóa cây kế thừa, chỉ cho phép các lớp con (`Success<T>`, `Failure<T>`) khai báo trong cùng một thư viện (*same library*).
- **Exhaustive Pattern Matching:** Trình biên dịch bắt buộc biểu thức `switch` phải kiểm tra đủ mọi trường hợp; báo lỗi ngay nếu bỏ sót nhánh.
- **Loại bỏ trạng thái nhập nhằng:** Đảm bảo không bao giờ tồn tại trạng thái không hợp lệ (*"vừa có dữ liệu vừa có lỗi"*).

2. Cú pháp Destructuring tinh gọn:

```
// Exhaustive pattern matching on sealed Result<T>
switch (result) {
  case Success(:final data):
    isLastActionSuccess = true;
    lastBookedTicket = data;
    location.deductSeats(type, data.amount);

  case Failure(:final error):
    isLastActionSuccess = false;
    statusMessage = error.message;
}
```

Xử lý Ngoại lệ trong Dart: Exception vs Error

1. **Phân biệt Exception và Error:**
   - **Exception:** Ngoại lệ dự đoán trước trong runtime (mạng, JSON). Ứng dụng **phải bắt và tự phục hồi**.
   - **Error:** Lỗi lập trình (`TypeError`, `RangeError`). Về nguyên tắc không nên bắt mà cần sửa code *(chỉ bắt tại biên để chống crash & log)*.
2. **Từ khóa `rethrow` & `StackTrace`:**
   - `catch (e, stackTrace)`: Bắt đối tượng ngoại lệ kèm vết ngăn xếp chi tiết.
   - `rethrow`: Đẩy lỗi lên trên mà vẫn **bảo toàn Stack Trace** gốc.
3. **Hai quy tắc vàng khi bắt ngoại lệ:**
   - Khối on cụ thể (on `AppException`) **phải đặt trước** `catch` chung để tránh bị nuốt mất ngoại lệ.
   - Khối `finally` **luôn thực thi** dọn tài nguyên kể cả khi khối lệnh trước đã gọi `return`.
4. **Cấu trúc đầy đủ try-on-catch-finally:**

```dart
try {
  final res = await apiCall();
} on AppException catch (e) {
  // Catch typed domain exception
  logError(e.message);
} catch (e, stackTrace) {
  // Catch unhandled & inspect trace
  logError('Unhandled: $e\n$stackTrace');
} finally {
  // Luôn chạy kể cả khi try đã return!
  closeConnection();
}
```

Cây phân cấp Custom Exception (AppException)

1. Triết lý phân cấp ngoại lệ tùy biến:

- Không ném Exception thô, luôn dùng lớp có code và message chuẩn hóa.
- Xây dựng lớp trừu tượng AppException triển khai (implements) interface Exception.
- Mỗi phân lớp phản ánh chính xác một nhóm lỗi nghiệp vụ trong ứng dụng.

4 Phân nhóm ngoại lệ chuẩn hóa

- **NET_ERR:** Mất kết nối mạng / máy chủ.
- **PARSE_ERR:** Lỗi giải mã dữ liệu JSON.
- **BIZ_ERR:** Vi phạm quy định số lượng vé.
- **UNKNOWN_ERR:** Sự cố hệ thống bất ngờ.

2. Triển khai kế thừa với Super Parameters (Dart 2.17+):

```
abstract class AppException implements Exception {
  final String message, code;
  const AppException(this.message, {required this.code});
  @override
  String toString() => '[$code] $message';
}

class NetworkException extends AppException {
  const NetworkException(super.message)
      : super(code: 'NET_ERR');
}
class DataFormatException extends AppException {
  const DataFormatException(super.message)
      : super(code: 'PARSE_ERR');
}
class BusinessLogicException extends AppException {
  const BusinessLogicException(super.message)
      : super(code: 'BIZ_ERR');
}
class UnknownException extends AppException {
  const UnknownException(super.message)
      : super(code: 'UNKNOWN_ERR');
}
```

Hàm bao bọc (Error Wrapper) & Result Pattern

1. Cơ chế đóng gói an toàn với `errorWrapper`:

- **Cổng Fail-safe ở tầng biên:** `catch (e, st)` bắt trọn mọi lỗi (kể cả Error) để tránh sập app, đồng thời ghi log chi tiết kèm `StackTrace`.
- **Chủ ý của `finally`:** Luôn thực thi dọn tài nguyên và log kết thúc phiên dù thành công hay ném lỗi.

```
// errorWrapper (phiên bản minh họa rút gọn)
Future<Result<T>> errorWrapper<T>(
    Future<T> Function() action) async {
  try {
    return Result.success(await action());
  } on AppException catch (e) {
    _recordError(e.toString()); // Log technical
    return Result.failure(e); // Domain typed
  } catch (e, stackTrace) {
    _recordError('Unhandled: $e\n$stackTrace');
    return Result.failure(const UnknownException(
        'Đã có lỗi xảy ra trong quá trình xử lý, vui lòng thử lại'));
  } finally {
    // Chủ ý: luôn chạy đóng phiên kể cả khi có lỗi
    logInfo('Wrapper', 'Hoàn tất phiên xử lý');
  }
}
```

2. Ma trận điều phối & thông báo thân thiện:

NGUYÊN NHÂN	MÃ LỖI	THÔNG BÁO UI
Số vé ≤ 0 / Hết chỗ	`BIZ_ERR`	*"Số lượng vé phải lớn hơn 0"* *"Không đủ vé tại phòng lab..."*
Mất mạng / Server sập	`NET_ERR`	Mất kết nối tới máy chủ
Payload JSON hỏng	`PARSE_ERR`	Dữ liệu máy chủ bị lỗi
Lỗi bất ngờ khác	`UNKNOWN_ERR`	Đã có lỗi xảy ra trong quá trình xử lý, vui lòng thử lại

Lưới an toàn toàn cục (Global Safety Net)

1. Hệ thống phòng thủ 2 lớp (Defense-in-Depth):

- **Lớp 1 (Framework):** `FlutterError.onError` bắt lỗi tầng framework (build/render widget, layout, gestures...).
- **Lớp 2 (Platform):** `PlatformDispatcher.onError` bắt ngoại lệ bất đồng bộ lọt ra ngoài luồng.
- Trả về `true` để báo đã xử lý, ngăn lan truyền mặc định; lỗi được gom vào Console và hiển thị an toàn trên UI.

Đồng bộ lỗi tức thì

Lỗi đẩy vào `errorLogs` của Controller và cập nhật lên Console Terminal màu đen trên giao diện.

2. Cài đặt thực tế trong `main.dart`:

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  final controller = BookingController();

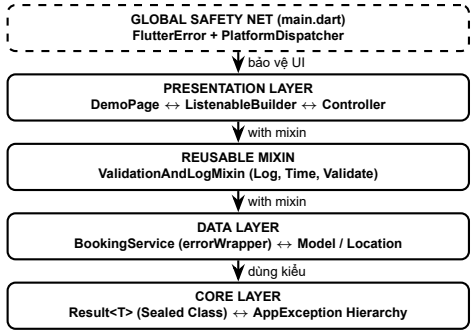
  // Layer 1: Catch framework UI render errors
  FlutterError.onError = (details) {
    FlutterError.presentError(details);
    controller.recordExternalError(
      'FlutterFramework', details.exception);
  };

  // Layer 2: Catch unhandled async exceptions
  PlatformDispatcher.instance.onError =
      (error, stack) {
    debugPrint('$error\n$stack');
    controller.recordExternalError(
      'PlatformDispatcher', error);
    return true; // Prevent app crash
  };

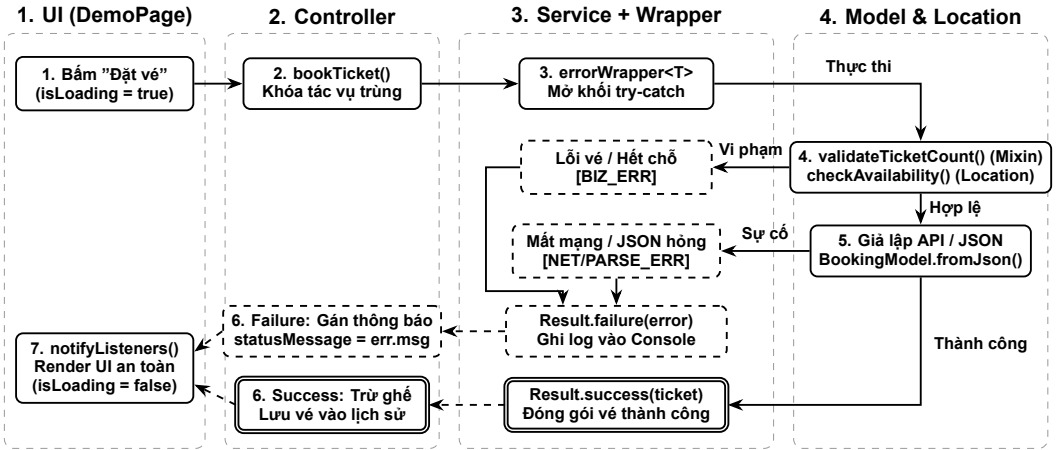
  runApp(MyApp(controller: controller));
}
```

Kiến trúc Phân tầng, State Management & Chống Race Condition

1. **Phân tầng kiến trúc rạch ròi:**
   - **Presentation Layer:** `DemoPage` ↔ `ListenableBuilder` ↔ `Controller`.
   - **Reusable Mixin:** `ValidationAndLogMixin` (dùng chung Controller & Service).
   - **Data Layer:** `BookingService` (`errorWrapper`), `Model`, `Location`.
   - **Core Layer:** `Result<T>` (Sealed Class), `AppException` (Cây ngoại lệ).
2. **Bất đồng bộ & Quản lý trạng thái:**
   - Áp dụng mô hình bất đồng bộ (`Future`, `async/await`, Event Loop).
   - `BookingController` phát thông báo qua `notifyListeners()`; giao diện dùng `ListenableBuilder` tái tạo chỉ các widget cần thiết.
3. **Chống Race Condition (Bấm liên tiếp):**
   - Chắn đầu tác vụ: `if (isLoading) return;`.
   - Khóa tương tác UI: `onPressed: isLoading ? null : _submitBooking`.



Sơ đồ Luồng dữ liệu



Nguyên tắc cốt lõi: Mọi nhánh lỗi hay thành công đều được chuẩn hóa qua `Result<T>` trả về Controller; tầng UI hoàn toàn sạch sẽ, không chứa `try-catch`.

Kịch bản Thực nghiệm & Bảng ca kiểm thử Live Demo

Ma trận kiểm thử nhanh tích hợp trên giao diện & ánh xạ kỹ thuật OOP:

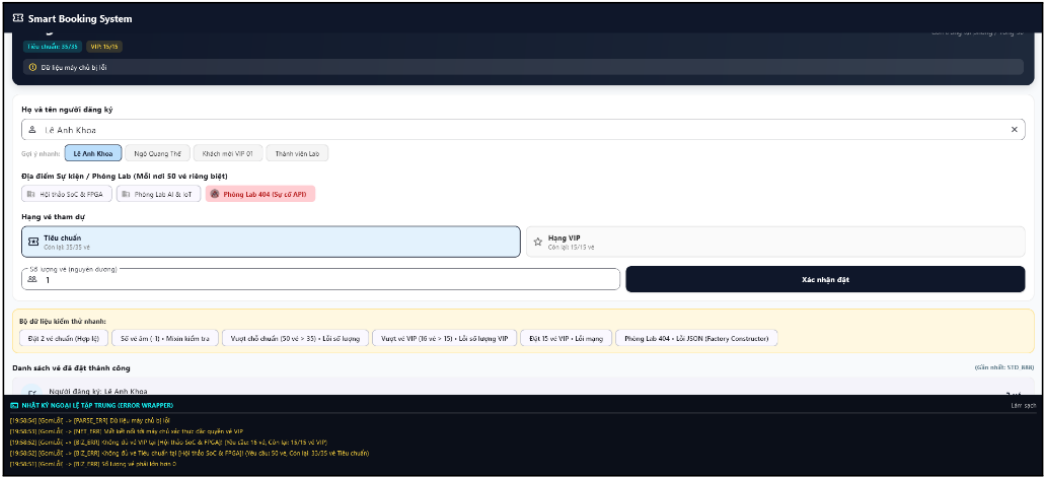
NÚT KIỂM THỬ	KỸ THUẬT OOP ÁP DỤNG	KẾT QUẢ GIAO DIỆN (UI BANNER)	MÃ LỖI
Đặt 2 vé chuẩn	Factory fromJson + Result.success (Sealed)	Đặt thành công 2 vé, khấu trừ ghế trống phòng lab	—
Số vé âm (-1)	Mixin: `validateTicketCount()`	Báo lỗi: *"Số lượng vé phải lớn hơn 0"*	`[BIZ_ERR]`
Vượt vé chuẩn (50 vé)	LabLocation: `checkAvailability()`	Báo lỗi: *"Không đủ vé Tiêu chuẩn tại [Hội thảo...]"*	`[BIZ_ERR]`
Vượt vé VIP (16 vé)	LabLocation: `checkAvailability()`	Báo lỗi: *"Không đủ vé VIP tại [Hội thảo...]"*	`[BIZ_ERR]`
15 vé VIP (Mất mạng)	`errorWrapper<T>` bắt ngoại lệ	Báo lỗi: *"Mất kết nối tới máy chủ xác thực đặc quyền..."*	`[NET_ERR]`
Lab 404 (Lỗi JSON)	Factory Constructor: `fromJson`	Báo lỗi: *"Dữ liệu máy chủ bị lỗi"*	`[PARSE_ERR]`

Nguyên tắc liên kết kịch bản kiểm thử với thiết kế OOP

- **Kiểm soát tại nguồn:** Lỗi bị chặn ngay tại tầng phát sinh (`validateTicketCount()`, `checkAvailability()`, `fromJson`).
- **Cách ly lỗi tập trung:** Cổng `errorWrapper<T>` chuẩn hóa ngoại lệ thành `Result.failure`, hiển thị an toàn trên UI.

Minh chứng Thực nghiệm: Toàn bộ Giao diện Ứng dụng

Giao diện thực tế ứng dụng Smart Booking System & Nhật ký gom lỗi Console:



Tổng kết Đề tài & Hướng phát triển Tương lai

1. Bốn kết quả cốt lõi đạt được:

- **Named & Factory Constructor:** Khởi tạo linh hoạt, chuyển đổi an toàn từ JSON bằng Pattern Matching Dart 3.
- **Mixin:** Tái sử dụng tính năng validate và log thời gian giữa các tầng mà không cần kế thừa phức tạp.
- **Generics + Sealed Class:** Một wrapper duy nhất cho mọi kiểu dữ liệu, ép xử lý triệt để nhánh kết quả.
- **Custom Exception + Result + Safety Net:** Ứng dụng không crash, thông báo thân thiện, gom lỗi tập trung.

2. Hạn chế & Hướng phát triển tương lai:

- **Hạn chế hiện tại:** Dữ liệu đang được giả lập trên RAM, chưa lưu trữ bền vững.
- **Hướng phát triển:**
  - Kết nối RESTful API thật (bắt `SocketException`, `TimeoutException`).
  - Lưu trữ dữ liệu cục bộ với SQLite hoặc Hive.
  - Mở rộng State Management chuyên nghiệp với Bloc / Riverpod.
  - Xây dựng hệ thống kiểm thử tự động (Unit / Widget Test) toàn diện.

Tài liệu tham khảo & Liên kết Dự án GitHub

Tài liệu kỹ thuật chính thức & Nguồn tham khảo chuyên môn:

1. **Dart Language Guide:** *Object-Oriented Programming, Mixins, Generics & Pattern Matching in Dart*, Dart Documentation.
   `https://dart.dev/language`
2. **Effective Dart:** *Guidelines for Style, Usage, and Design in Dart*, Dart Documentation.
   `https://dart.dev/effective-dart`
3. **Handling Errors in Flutter:** *Error handling, `FlutterError` & `PlatformDispatcher`*, Flutter Documentation.
   `https://docs.flutter.dev/testing/errors`
4. **Kho lưu trữ mã nguồn Đề tài (GitHub):** Dự án mã nguồn mở trên Windows Desktop Native & Flutter SDK.
   `https://github.com/KhoaLe246205/cross-platform/tree/main/oop_error_handling`

XIN CHÂN THÀNH CẢM ƠN!

QUÝ THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE